A Custom Lightweight Convolutional Neural Network Architecture for Edge-Based Inference using Arduino.

Ngoc Lam Nguyen
Narigela Grishma Reddy
Revanth Puvaneswaran
Jatinkumar keshabhai Parmar

EPITA Graduate School of Computer Science

Advisor
Alaa Bakhti

Date: 15/07/2025

Declaration of own work

This work has been composed solely by us.

This work has not been accepted in any previous application for a degree or diploma, by (all student full names go here), or anyone else.

The work of which this is a record has been wholly done by us.

All extracts, including those created by generative AI, have been distinguished by quotation marks and the sources of our information have been specifically acknowledged and documented with in-text citations and in the bibliography.

All prompts with responses for any use Generative AI must be included in a separate MS Word document, clearly referencing its appearance in the bibliography and allowing the reader to easily associate the extract you used with the prompt. Here is how APA cites Generative AI: https://apastyle.apa.org/blog/how-to-cite-chatgpt

Date: 15/07/25

Name: Ngoc Lam Nguyen

Signature:

Name: Narigela Grishma Reddy

Signature:

Name: Revanth Puvaneswaran

Signature:

Name: Jatinkumar keshabhai Parmar

Signature:

Acknowledgements

We would like to express our genuine gratitude to our mentors, faculty members, and fellows who provided important insights and support throughout this project. Special thanks to our project guide for their informed perspectives, which were used in navigating the complexities of ethernet edge and lightweight neural network design. We are also thankful for the open-source community and researchers whose work in CNN optimization and TinyML has a strong impact on our research. Lastly, we appreciate the collaborative efforts of our team members, whose dedication and hard work have been impressive in progressing this project from basic to practical implementation.

Abstract / Executive Summary

Over the past years, deep learning models have shown phenomenal success in various computer vision tasks. However, deploying these models on edge devices remains difficult due to their algorithmic and memory stipulation. This project aims to design and originate a custom lightweight Convolutional Neural Network (CNN) architecture specifically enhanced for limited resource environments like Arduino based TinyML kits. By targeted optimizations, the proposed model strives for balanced accuracy with processing capability, enabling real time deduction on edge devices. The research includes evaluating the custom structure against current lightweight models like MobileNetV3 and ShuffleNet, applying techniques namely Quantization-Aware Training (QAT) and Post-Training Quantization (PTQ). The final goal is to implement the optimized model on Arduino, making workable usability for active applications with nominal resource consumption.

Contents

Acknowledgements 1
Abstract / Executive Summary 3
1. Introduction 6
1.1. Motivations 6
1.2. Context of the project 6
1.3. Objectives 7
1.4. Overview of the Thesis 7
2. Literature Review 8
3. State of the Art 17
3.1. MobileNet Architecture 17
3.2. ShuffleNet Architecture 18
3.3. Why we choose ShuffleNetV3 as our transfer learning model 20
3.4. Quantization 21
3.5. Our Custom CNN model 23
4. System Architecture 24
4.1. System Architecture anticipated 24
4.1.1. Edge Device Layer 25
4.1.2. Backend and Model Layer 25
4.1.3. User Interface Layer 26
4.2. System Architecture updated 26
5. Methodology 26
5.1. Methodology anticipated 26
5.1.1. Transfer Learning model 26
5.1.2. Quantization 27
5.1.3. Custom CNN model 28
5.2. Methodology updated 31
6. Results 31
6.1. Development anticipated 31
6.1.1. Transfer Learning model 31
6.1.2. Quantization 34
6.1.3. The effect of lighting and contrast in our results 38
6.1.4. Our Custom CNN 40

6.2. Development updated 43
7. Conclusions 43
7.1. Summary of Work 43
7.2. Key Contributions 44
7.3. Future Work 44
7.4. Final Remarks 44
Bibliography/References 45
Appendices 49

1. Introduction

1.1. Motivations

The high growth of Artificial Intelligence (AI) and deep learning has changed computer vision, natural language processing, and other domains. Although these advancements, utilizing complex models on edge devices, such as emended systems and low-power platforms, remain a great challenge. These devices commonly used on the Internet of Things (IoT), have severe constraints on processing power, memory, and energy consumption.

This gap between high-performing models and low resource hardware motivates the need for custom lightweight neural network architecture. Such developments enable actual AI capacities on devices without dependence on cloud computing, which enhances privacy, lowers delay and increases operational efficiency. Therefore, this research aims to bridge this gap by designing a lightweight Convolutional Neural Network (CNN) improved for Arduino-based TinyML kits, granting to the wider usage of AI at the edge.

1.2. Context of the project

This project fibs at the interchange of deep learning, computer vision, and edge computing. Edge devices like Arduino ML kits are becoming more relevant in scripts, cartable low-power AI solutions such as smart monitoring systems, handy instruments and IoT devices.

While standard models like MobileNetV3 and ShuffleNet simplified lightweight solutions, they are not always amended for certain tasks or the software restrictions of microprocessors. This project appeals to the gap by creating a task-specific lightweight CNN that may play efficient image classification using the CIFAR-10 dataset, broadly known standard for minor vision tasks.

By focusing on fixed systems and real-time operation, the project helps the field of TinyML and AI gear extension.

1.3. Objectives

- To design a custom lightweight CNN architecture crafted for image classification tasks on edge devices.
- To upgrade framework for actual guess on Arduino ML kits while holding decent accuracy levels.
- To access model optimization techniques such as Quantization Aware Training (QAT) and Post Training Quantization (PTQ) to play down cyber and memory terms.
- To pattern the blueprint against existing lightweight models to review its execution in terms of accuracy, speed, and resource usage.
- To develop and exhibit the model on an Arduino TinyML kit for validation.

These objectives above will refer to the research aims to enable automated solutions in screenplay where cloud reliance is dreamy due to connectivity and privacy worries.

1.4. Overview of the Thesis

- Chapter 2: Reviews literature on lightweight CNNs and smart edge
- Chapter 3: Analyze current state of the art models.
- Chapter 4: Details system design and methodology.
- Chapter 5: Describes experimental setup and implementation.
- Chapter 6: Presents results and performance evaluation.
- Chapter 7: Concludes with key findings and future work.

2. Literature Review

In the world of image processing, convolutional neural network (or CNN for short) is the current trendy architecture. From the starting point of AlexNet in 2012, to a single-stage detector that specializes in object detection like YOLO, CNN has made some impressive strives. It “has rekindled interest in deep learning among academics” (Zhao et al., 2024, p.37). However, most recent works are only exploring how we can make the model more accurate. One aspect that is often ignored is the size of the models. With small devices are appearing more and more around us, like IoT devices, which expected to be around “40 billion IoT devices by 2030” (Sinha, 2024), many of which cannot support a complex model by various reasons, there is a need for downsizing the CNN model to fit that requirement. Which is why we are proposing a Custom Lightweight Convolutional Neural Network Architecture for Edge-Based Inference using Arduino.

A Convolution Neural Network (CNN) is a special type of neural network designed for tasks associated with spatial data, such as images. The concept of neural networks, where each node is called a neuron and connected to the next layer, is that it uses a more effective approach by dividing weight through CNN filters or cores. These filters slip on input data in a process called kernel, which captures local patterns such as edges, textures, and shapes. Specific CNN architecture includes several main teams. The performance of unique hybrid CNN models typically improves over later steps. (Tahir, 2024, p. 1) It begins with an input team receiving raw data by affects layers that uses the filter to remove it. An activation layer, using the Relu function adds non-economically, enables the model to learn complex patterns. Pool layer, which reduces the dimensions of the data, improves calculation efficiency to increase the convenience of the data. By stacking many fixed, activation and pool layers, CNN teaches hierarchical

functions from simple to complex structures. This layered approach allows CNN to identify and classify visual patterns with high accuracy (Sarafraz I, 2025, p. 1)). Theirability to detect features regardless of sharing parameters and detecting features makes them very effective for image classification, object detection and other pattern recognition tasks in areas such as health care, safety and autonomous systems.

Edge gadgets inside the Internet of Things (IoT) are essential components which serve as the interface between the physical global and digital networks. These devices acquire methods to transmit statistics from sensors to centralized systems or cloud structures. Common examples consist of microcontrollers like Arduino, that are preferred for low cost, compact length, and power performance. Typically, edge gadgets are designed to be small and battery-powered, making them appropriate for deployment in far-flung or aid-restrained environments. Due to their restrained processing talents, these gadgets often perform preliminary facts processing tasks, which include filtering or aggregation, earlier than transmitting the information to greater effective structures for similar analysis. Numerous organizations and universities have dedicated themselves (Manickam, 2024). This technique reduces the quantity of data transmitted, conserves bandwidth, and complements the general performance of the IoT system. The integration of area computing into IoT systems permits real-time statistical processing, which is vital for packages requiring immediate responses, together with environmental tracking or commercial automation. Edge devices, networks, communication units, and cloud platforms are all part of the proposed distributed hierarchical data (Malaysia, 2024, p. 4) fusion architecture for IoT networks. By processing records domestically, part devices can lessen latency, enhance reliability, and hold capability even when connectivity to important systems is intermittent.

As the number of IoT units increases, it becomes important to distribute intelligent treatment functions on direct edge equipment. Traditional deep learning models, especially CNN-based architecture, are known for their high accuracy in vision tasks, but are calculated to be expensive. This makes them unsuitable for placement on edge equipment, which usually suffers from limited calculation power, memory barriers, and strict energy budget. The task introduces MicroVit, a light vision transformer that integrates CNN components to enable effective and correct visual processing on edge platforms. IoT devices can guess real time without relying on cloud-based calculations, one of the main benefits of entering such customized CNN-based models. It reduces delays, increases privacy, and reduces the use of bandwidth. CNN-integrated models that MicroVit also enable local decisions. For applications such as industrial monitoring and environmental sensations, it is important for responsibility for real time. In addition, paper shows that microVit, despite its simplified architecture, receives comparable or better performance for traditional light models such as Mobilentv2 and MobileVit, while improving the speed up to 3.6 × and increased energy efficiency by 40%. In the initial stages, the model balances the model's accuracy and the calculation load using techniques such as (ESHA) and depth of convention layers. This covered approach allows CNN opportunities to be used effectively in inaccessible scenarios due to resource constraints. The result is scalable, cost-effective, and powerful equipment for the next generation IoT system.

What could be more challenging than putting CNN in edge devices as AI needs real time sensory data to assist in every possible way in many industries. Moreover, the optimization of deep-learning models results in reduced resource consumption, making them ideally suited for deployment on resource-constrained IoT devices. This efficiency enables a streamlined and

effective utilization of resources, underscoring the practicality and sustainability of incorporating deep learning into IoT systems (Electronics 2023). There are many constraints that come with achieving this, including the balance between speed and accuracy, making the deep learning models faster without losing much of their accuracy. Reasonably shrinking in terms of number of parameters resulting model in a smaller scale and perform quantization on them. Quantization has often been used to efficiently optimize CNNs for memory and computational complexity at the cost of a loss of prediction accuracy (Goyal et al., 2021). When it comes to deployment after the quantization techniques, a gradual loss in accuracy can be expected. Acquiring real time data from edge devices must be protected with data protection policies. In industries like healthcare, smart homes, and finance, sending sensitive images or videos off-device raises privacy concerns. Local processing helps meet GDPR and similar regulations by keeping data secure (Edge AI Vision). As models become lightweight and the hardware becomes more reliable the consumption of power is very low. Most deep object models demand too much Central Processing Unit (CPU) power and cannot be used on edge devices, even though many object detectors attain outstanding accuracy and carry out inference in real-time. The necessity to execute backbone designs on edge devices with constrained memory and processing power stimulates research and development of deep learning-based light-weight object identification models (Payal Mittal).

Deep learning models have been outperforming humans in every major action such as fault detection and many more applications. All these models run on a huge computing power to work in a real-world setting. Here comes the constraints of how about optimizing them into smaller devices such as edge devices. Thus, the intro to the lightweight architecture which

includes backbone, neck and detector. For instance, we provide an input image with pre-fixed constant shape or a pyramid which creates the copies with vertically stacked version. Then the image is processed by the convolutional layer. Then these set of layers identifies the relevant features from the image. A feature map has been extracted from the given input image which is nothing but a set of dominant features from the original input image. Some of the existing lightweight architectures such as MobileNet, ShuffleNet, PeleeNet and many more and these architectures come with dedicated versions based on their recent updates. Deep learning-based models for image processing advanced and effectively outperformed more conventional methods in terms of object classification (Mittal 2024). All these have been combined together to find the precise image such as output. Then it also assists in merging features from different layers which helps in detecting smaller or obscured objects (Electronics, 2023). In the end, the detector's head describes the precise items.

The deployment of lightweight CNNs on resource-constrained edge devices requires an approach that maintains accuracy while reducing computational complexity and model size. The "Building Efficient Lightweight CNN Models" paper introduces a new dual-input-output architecture, where two identical submodes are trained simultaneously, one on original data and another on augmented data and concatenated their outputs. With 14,862 parameters this model achieves 99% accuracy on MNIST dataset (Nathan, 2025, p. 3). This shows how architectural design can outperform traditional compression methods like pruning and quantization. Similarly, EdgeNeXt finds a hybrid CNN-Transformer model featuring a Split Depth-wise Transpose Attention (SDTA) encoder which reduces FLOPs by 28% while maintaining 79.4% accuracy on ImageNet with 5.6M parameters (Maaz et al., 2022, p. 10). This paper emphasizes

the importance of linear complexity attention mechanisms for mobile vision applications. In “MicroViT: A Vision Transformer with Low Complexity Self Attention for Edge Device” takes a different approach by optimizing Vision Transformers through its Efficient Single Head Attention (ESHA) mechanism, which reduces computational complexity by 46.8% compared to standard ViTs while achieving 3.6× faster inference speeds. DP-Net enhances MobileNetV3 through partial residual structures and dual-path feature extraction, reduce FLOPs by 55.1% with minimal accuracy loss (Zhao et al., 2022, p. 12). The "Lightweight Deep Learning Models" survey compares compression techniques and shows that hybrid approaches combining pruning (up to 50% size reduction), quantization (4× memory savings), and knowledge distillation achieve the best results (97% accuracy at 3.3MB) (Aminu Musa, 2025, p. 14). Common themes across these papers include the superiority of architectural innovations over post-hoc compression, the importance of task-specific optimization, and the need for hardware-conscious design. The most effective solutions combine multiple approaches - for instance, EdgeNeXt's hybrid architecture with hardware-friendly operations, or the survey's recommended combination of pruning, quantization, and distillation.

The field of lightweight CNN research is characterized by several ongoing debates that reflect fundamental trade-offs in model design. The most important is the accuracy-efficiency trade-off, which is shown by EdgeNeXt's impressive 79.4% ImageNet accuracy with lower computation versus MicroViT's more efficiency (3.6× faster than MobileViT) at 72.6% accuracy. The "Building Efficient Lightweight CNN Models" paper reveals how this trade-off varies by dataset, with their model achieving 99% on MNIST but only 65% on CIFAR-10, which highlights the challenge of maintaining performance across different complexity levels (Nathan, 2025, p. 15). According to

“Lightweight Deep Learning Models for Edge Devices—A Survey” compression methods have another debate: structured versus unstructured pruning (hardware compatibility vs. higher compression) (Aminu Musa, 2025, p. 3), and quantization-aware training versus post-training quantization (accuracy vs. simplicity) (Aminu Musa, 2025, p. 8). The “TinyML” papers introduce another strategy deployment of pre-trained models with adaptive on-device learning approaches like TinyOL, weighing the benefits of fixed efficiency against the flexibility of continuous learning (Rakhee Kallimani, 2023, p. 7). Another debate is to develop universal frameworks or optimize for specific hardware platforms. This is particularly relevant given the diversity of edge devices, from ARM Cortex MCUs to NVIDIA Jetson boards. This paper also showing different techniques about model development: some building efficient architectures from scratch (like EdgeNeXt and DP-Net), while others focus on compressing existing models. The debate is all about creating general models that perform across many tasks or optimized solutions for specific applications.

Recent lightweight models have produced several advanced architectures and optimization techniques. “EdgeNeXt” introduced a CNN-Transformer based hybrid design, it has achieved the state-of-the-art performance over multiple benchmarks - 79.4% accuracy on ImageNet, 27.9 mAP on COCO detection, and 80.2 mIoU on Pascal VOC segmentation - while reducing computation complexity by 35-38% compared to other similar models (Maaz et al., 2022, p. 11). EdgeNeXt’s SDTA encoder a big improvement in making attention mechanisms for more efficient mobile vision (Maaz et al., 2022, p. 5). “MicroViT” offers another way with its Efficient Single Head Attention (ESHA), which reimagines transformer attention for edge devices, it has 3.6× faster inference than MobileViT with good accuracy (Novendra Setyawan, 2025, p. 4). DP-Net's

fine-tuning to MobileNetV3 through partial residual structures and dual-path feature extraction shows how careful architectural modifications can give notable efficiency gains (55.1% FLOP reduction) with minimal accuracy loss (Zhao et al., 2022, p. 12). The "Lightweight Deep Learning Models" survey's hybrid approach combines the best aspects of pruning, quantization, and knowledge distillation into a unified framework that achieves 97% accuracy at just 3.3MB, that highlights the power of integrated compression techniques (Aminu Musa, 2025, p. 14). TinyML research has created special small models, like Tiny Transducer for recognizing speech and Tiny RespNet for analyzing breathing (Rakhee Kallimani, 2023, p. 6). The paper called "Building Efficient Lightweight CNN Models" introduced a new design with two inputs and outputs, showing that small models made for specific tasks can work better than general models (Nathan, 2025, p. 19). Later models like “EdgeNeXt and MicroViT” build upon earlier work (MobileNetV3, Vision Transformers) while introducing new mechanisms to solve specific efficiency problems. These advancements make it easy to apply deep learning models on small, low-power devices, while keeping good accuracy and performance.

With that being said, as of right now, in the field of Lightweight architecture for edge devices, particularly deep learning models, research efforts are mainly spent on answering 3 questions. The biggest question is how we can put more complex models into edge devices that are constrained in terms of memory and/or processing power like IoT devices, smartphones, and embedded processors (Mittal, 2024). According to the article “A comprehensive survey of deep learning-based lightweight object detection models for edge devices”, “The research community has long worked to identify the best accuracy detection models through advanced architectural searches, as developing the deep learning-based lightweight network architecture

is a difficult procedure. When using these models in edge devices, such as high-performance embedded processors, the question arises regarding usage of high-end innovative applications with fewer resources." (Mittal, 2024, p. 2). Another big question to be answered is the trade-off between performance and resource usage. Generic deep learning models usually have high computational complexities, and require extensive resources, large data processing pipelines, of which edge devices do not have. Researchers found that if the model compressed beyond a certain point, the accuracy of it is decreasing, compared to the full-sized model (Aminu Musa et al., 2025). Finally, in my opinion, there are lots of technical challenges in this field of research, which keep it far from real-world applications. There are a lot of debates about which method of compression is the best: quantization, pruning, and knowledge distillation (Aminu Musa et al., 2025, Figure 1). Some researchers suggest combining multiple compression techniques into hybrid models. This way could lower performance degradation and still achieve a good parameter reduction. As a bonus point, according to "TinyML: Tools, Applications, Challenges, and Future Research Directions ", "The absence of standardization is one of the main problems due to which it is challenging for developers to generate cross-platform compatible solutions." (Rakhee Kallimani et al., 2023, p.10). Since 2021, research on lightweight models has experienced a notable increase, as observed by the author (Aminu Musa et al., 2025, Figure 6). This surge is likely driven by the growing demand for sophisticated models on resource-constrained edge devices.

Gap:

We are keen to explore one of the gaps in computer vision model research, which is to create a lightweight model capable of running on limited resources devices. With this problem just

starting to be recognized, there are going to be a lot of difficulties in developing such a model. One of which is that there are no standard criteria to compare models (Maaz et al., 2022) (Mittal, 2024). There are top-1 accuracy (Mittal, 2024, Table 7), “parameters and multiplication-addition (MAdds)" for computational complexity (Maaz et al., 2022, Table 3). Moreover, the model still has to balance the need between accuracy and efficiency. But by the end of our research, hopefully we can improve one of the criteria that is being used to compare lightweight models.

3. State of the Art

Current State of the Art for TinyML models are MobileNetV3 and ShuffleNetV2. In this section, we will dive into both models to see which one is better suited for our needs. We also propose our own small, robust model. Our model, inspired by what we learnt in classes, will perform object detection tasks with the constraint inside a microprocessor, such as Arduino.

3.1. MobileNet Architecture

For the transfer learning model, there are two models that we think are state of the art (SOTA); MobileNetV3 and ShuffleNetV2. The MobileNetV3, as the name suggests, is the third iteration of the MobileNet architecture. The core element is the inverted residual structure with linear bottlenecks, introduced in MobileNetV2. “This module takes as an input a low-dimensional compressed representation which is first expanded to high dimension and filtered with a lightweight depthwise convolution” (Sandler et al., 2019). The narrow layers, known as bottlenecks, do not have non-linearity in them. Residual connections are applied directly between the thin bottleneck layers. This inverted design is more memory efficient and performs

slightly better than traditional residual connections (Sandler et al., 2019). What is new in the MobileNetV3 compared to the older model are the Squeeze-and-Excitation (SE) modules and a new novel H-Swish Non-linearity. SE modules are placed after the depth wise filters in the expansion layer, allowing attention to be applied to the largest feature representation (Howard et al., 2019). The size of the SE bottleneck is fixed to be 1/4 of the number of channels in the expansion layer (Howard et al., 2019), which improves accuracy with a modest increase in parameters and no noticeable latency cost. For the H-Swish, it is based on the swish function of the old model. The standard Swish function is defined as: x * σ(x). However, the sigmoid function is computationally expensive on mobile devices. In H-Swish function, the researchers replaced the sigmoid function with $\frac{RELU6(x+3)}{6}$, also known as the piece-wise linear hard analog (Howard et al., 2019). This function is based on RELU6 function, which is faster to compute and more friendly for quantization. MobileNetV3 also leverages two complementary search techniques. They used platform-aware NAS to search for the global network structures by optimizing each network block (Howard et al., 2019) and NetAdapt algorithm for layer-wise fine-tuning, which adjusts individual layers to minimize the ratio of latency change to accuracy change. These changes, with the redesign of the costly layer in the network, make MobileNetV3 models outperform MobileNetV2 and other state-of-the-art models like MnasNet and ProxylessNAS in terms of accuracy-latency trade-offs on mobile devices, as shown in Figure 1 (Howard et al., 2019).

3.2. ShuffleNet Architecture

The ShuffleNet V2 model is specifically designed for mobile and embedded vision applications, with a primary focus on achieving high actual speed (low latency) while maintaining accuracy.

According to Figure 3 (Ma et al., 2018), a ShuffleNetV2 unit has 2 branches. At the beginning of each ShuffleNetV2 unit, the input feature channels (c) are split into two distinct branches; one is for identity mapping, the other is for processing the features as normal. This design promotes feature reuse, similar to DenseNet, but with a more efficient, exponentially decaying reuse pattern that avoids redundancy.

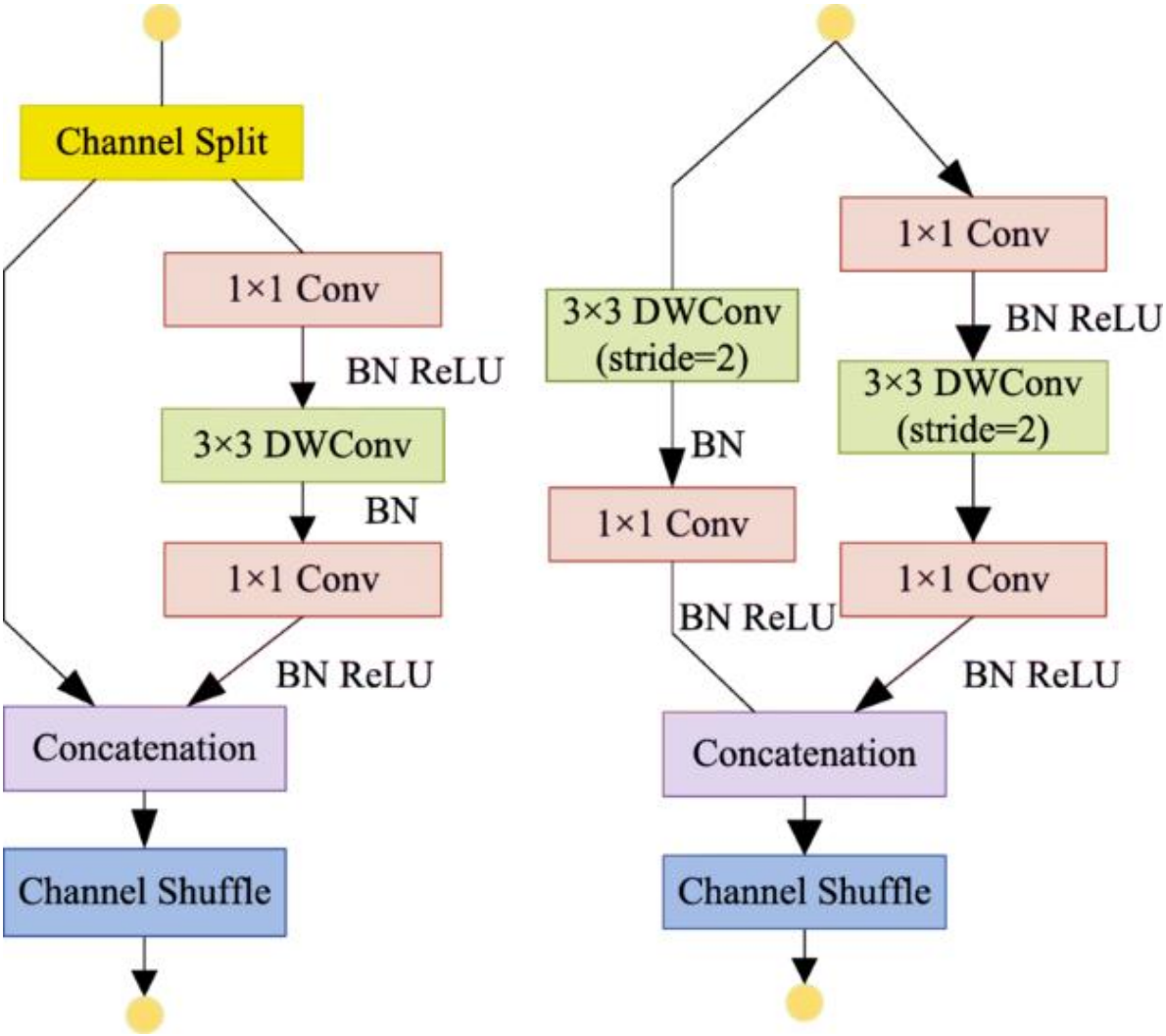


Figure 1: ShuffleNet Architecture

In the convolutional branch, the researchers proposed a 1x1 convolution, followed by a 3x3 depthwise convolution, and finally another 1x1 convolution. Unlike the first version, the 1x1

convolutions in this branch are no longer group-wise (Ma et al., 2018). This is done to avoid putting more emphasis on memory access, thus reducing memory access costs. Element-wise operations, including ReLU activation function and depthwise convolutions, are primarily contained within this processing branch. As a result, all of those “expensive” operations are done on only half the features. After processing in the convolutional branch, the outputs from both the identity branch and the convolutional branch are concatenated. This concatenation maintains the overall channel count, keeping an equal channel width (Ma et al., 2018). A channel shuffle operation, inherited from ShuffleNetV1, is applied to enhance the network's representational power through information exchange and communication between the two distinct branches. However, the "Add" operation from the shortcut connection in ShuffleNetV1 is removed in ShuffleNetV2. For units that perform spatial down-sampling (reduce the spatial resolution of the feature maps by using a stride of 2), the initial channel split operation is omitted. Instead, one branch performs a 3x3 average pooling with a stride of 2, while the other branch employs a 3x3 depthwise convolution with a stride of 2 (Ma et al., 2018). This refined structure allows ShuffleNetV2 to achieve significant gains in both accuracy and actual inference speed compared to previous lightweight models by carefully balancing computational complexity with hardware-aware efficiency considerations, as shown in Table 8 (Ma et al., 2018)

3.3. Why we choose ShuffleNetV3 as our transfer learning model

In order to benchmark our model performance, we would need to fine-turn a backbone architecture. Our pick for backbone architecture is between MobileNetV3 and ShuffleNetV2. While according to the paper Which Backbone to Use: A Resource-efficient Domain Specific Comparison for Computer Vision, “ShuffleNet is a better choice than MobileNet when very light-

weight models are needed" (Jeevan & Sethi, 2024, chapter 6), we have decided that MobileNetV3 is better for us. The reason for this is that we believe that MobileNetV3 is slightly newer than ShuffleNetV2. It also leverages Neural Architecture Search (NAS), Squeeze-and-Excitation (SE) blocks, and h-swish activation (Li et al., 2022, Figure 4). It's a strong choice when you need a balance between performance and efficiency, especially for tasks like image classification and object detection on mobile devices.

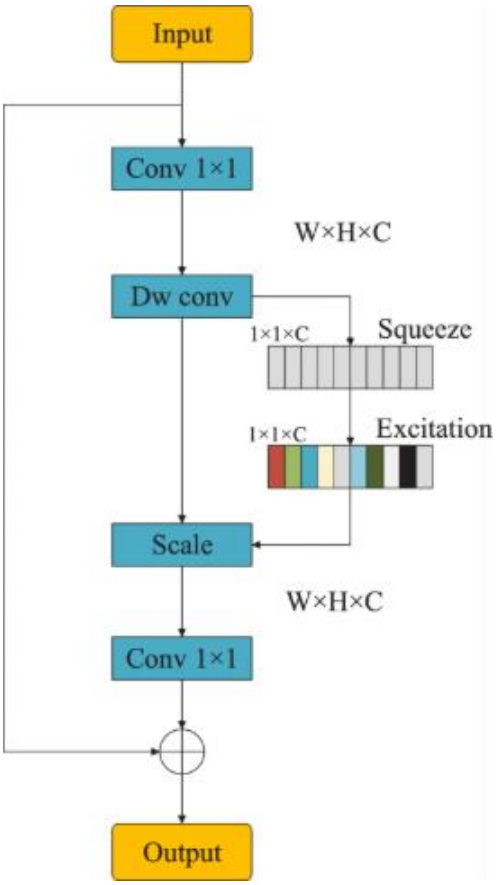


Figure 2: MobileNetV3 Cell

3.4. Quantization

Quantization technique is used in deep learning to optimize models by reduce the precision of numbers that are utilized to represent the model's parameters and its computations. In

traditional neural networks, 32-bit floating point (FP32) numbers are being used for weights and activations. It converts these numbers to lower-precision formats in 16-bit floating point or 8-bit integer, which makes the model smaller and faster. There are various approaches to performing quantization. Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT) are the main categories of quantization. PTQ is applied after completing model training and does not require any retraining; that makes it a simpler and more efficient option with a single function call in PyTorch or TensorFlow. On the other side, QAT is performed during the model training process, which makes it more complex but achieves higher accuracy. There are different types of quantization techniques, like dynamic quantization, where only weights are quantized ahead of time and activations are quantized at runtime dynamically. In static quantization, weights and activation are both quantized in advance, that requires calibration with a representative dataset. In weight-only quantization, activations are in full precision, and weights are quantized. Quantization has several benefits, that make it a valuable technique for compressing deep learning models. One of the first advantages is reduced model size—INT8 models, these models are typically four times smaller than their FP32, which significantly simplifies deployment on edge devices. Quantized models are faster during inference due to the efficiency of integer operations, particularly on hardware designed to support integer operations. The reduced model size also benefits lower memory bandwidth requirements, which is critical for maintaining performance real-time applications. Hence, quantization allows models to run on a wide range of resource-constrained devices like microcontrollers and smartphones while facilitating deployment in various environments. Quantization also has some drawbacks too like quantized model shows lower accuracy than originals. Especially in models which are sensitive

to small changes in weights and activations. Another challenge is hardware compatibility, as some of the systems do not support efficient integer computations that can limit performance. Moreover, some layers or operations are not easy to quantize, they are more complex. Debugging quantized models are also more complex, as compared to full-precision models as numerical errors and performance issues may not be as easy to identify and resolve.

3.5. Our Custom CNN model

Nowadays combining artificial intelligence with small devices is becoming more popular, but it does come with some restrictions related to model size and computation. This project is to create a simple convolutional neural network from scratch that can recognize images from the CIFAR-10 dataset and able to run on edge devices like Arduino. The main goal of this project is to make the model accurate and small enough to work on edge devices with low memory and processing power.

The following steps are,

- Building a CNN from scratch to classify CIFAR-10 images.
- Train and test the model with preprocessing operations like data augmentation.
- Checking the model with the accuracy and confusion matrix.
- Finally converting it to TensorFlow Lite so it can run on edge devices.

4. System Architecture

4.1. System Architecture anticipated

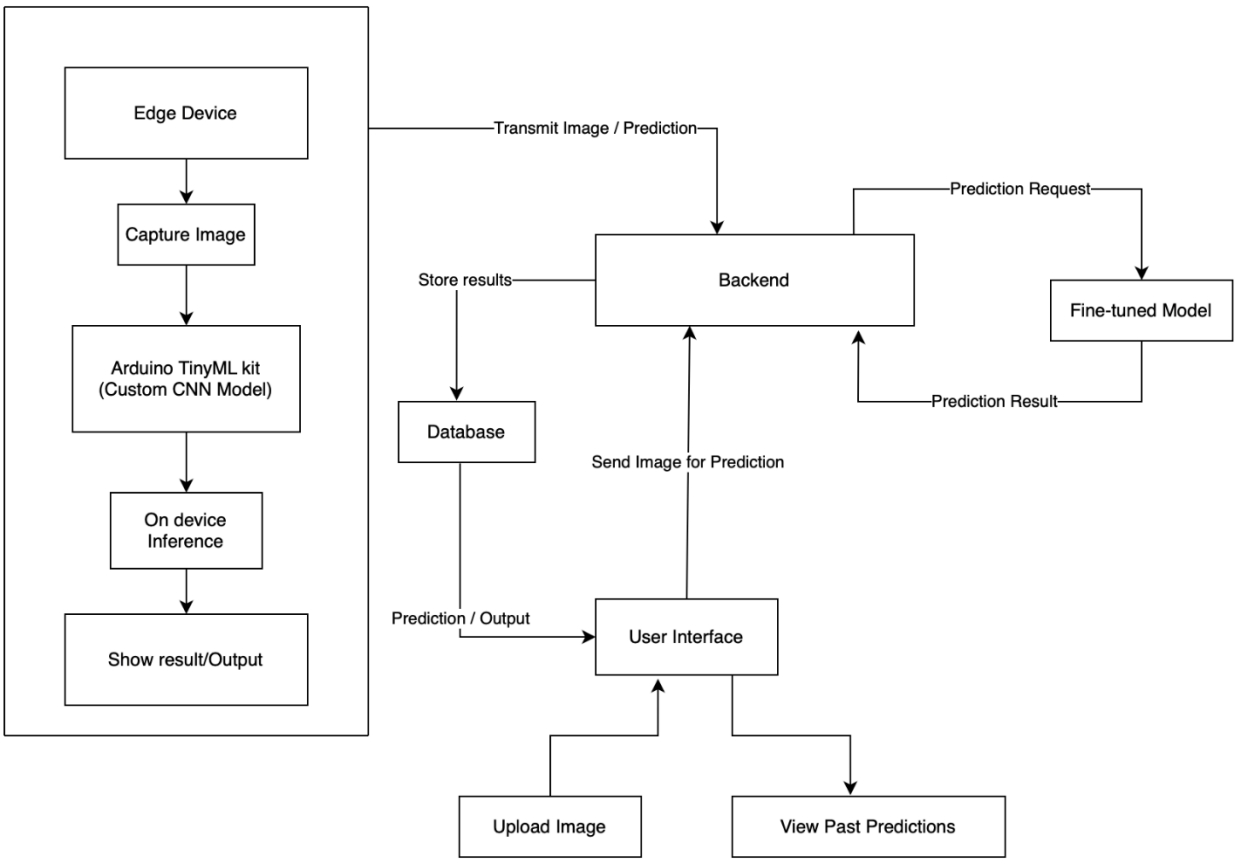


Figure 3: System Diagram

The system architecture shows a complete end-to-end pipeline for our projects to perform image classification using a TinyML-powered edge device integrated with a cloud-based backend and a responsive user interface. This architecture allows for efficient local inference while also enabling enhanced prediction capabilities through a fine-tuned server-side model. The system is composed of three main layers:

4.1.1. Edge Device Layer

In this layer we deployed custom lightweight CNN model on Arduino-based TinyML kit, this device can capture real-time image using onboard camera and make predictions on deployed model. This layer includes the Arduino-based TinyML kit, which serves as the primary data acquisition and local inference unit. On-device inference ensures that predictions can be generated with minimal latency and without reliance on constant internet connectivity. The prediction result can be immediately displayed to the user or sent to the backend for centralized logging, extended analysis, or refinement using a more powerful model.

4.1.2. Backend and Model Layer

The backend acts as the central communication hub of the system. It receives both images and prediction results from the edge device and user interface. When higher accuracy or additional model capabilities are required, the backend sends the image to a fine-tuned model hosted on a server. This model, developed by fine tuning MobileNetV3 and trained with a CIFAR10 dataset, provides refined prediction results. Once computed, these predictions are returned to the backend and stored in the database.

The backend is developed using FastAPI, which can handle requests from the edge device and user interface. It sends prediction requests to the fine-tuned model and receives output from the model, and stores all data, including predictions and timestamps, into database a centralized (PostgreSQL).

4.1.3. User Interface Layer

The user interface enables users to interact with the system by uploading images for prediction and viewing past prediction results. UI built using a Streamlit framework, where Users can upload single image directly and make predictions on fine-tuned model, and they can also view past predictions history from a database.

4.2. System Architecture updated

5. Methodology

5.1. Methodology anticipated

5.1.1. Transfer Learning model

We have chosen to use the pretrain MobileNetV3 from PyTorch library. We also chose the small version because of Adruino's limitations compared to other microprocessors in the market. Also, to keep the model as simple as possible, we only modified the last fully connected layer into the 10-class shape that we want. I achieved that by self.model.classifier[3] = nn.Linear(1024, 10). As we will discuss more detailed later on, we found out that lighting and contrast affects the model's performance a lot. For example, the accuracy of the model is reduced to around 0.66 if we have a 10 percent increase in brightness and contrast. To combat this, we also introduced a lighting layer that can be modified by the user. However, we did not include this in the final model, instead opt to make it a feature for web prediction. The reason for this choice is that we cannot predict the exact environment that the camera and our microprocessor would be in. Overall, our transfer learning model, although small, is still not of the size that we wanted to put in our microprocessor. Furthermore, the amount of non-trainable parameters is still huge,

owing to the structure of the MobileNetV3. It forces us to think about a more aggressive way to reduce the size of the model or redesign the network from the start.

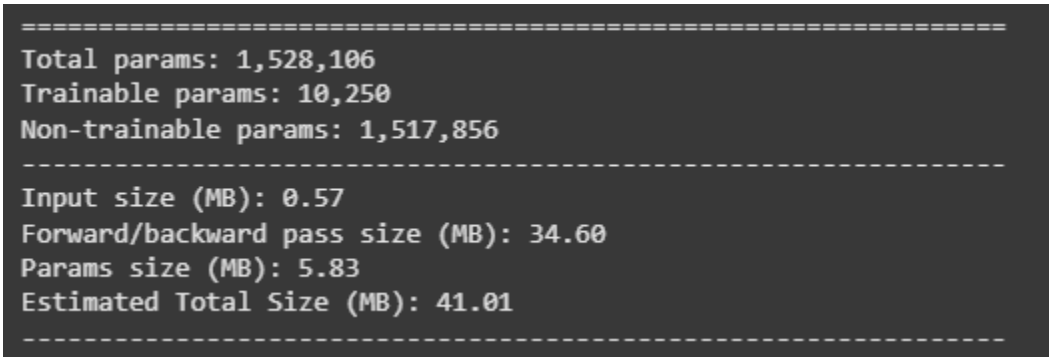


Figure 4: Transfer Learning Model Size

5.1.2. Quantization

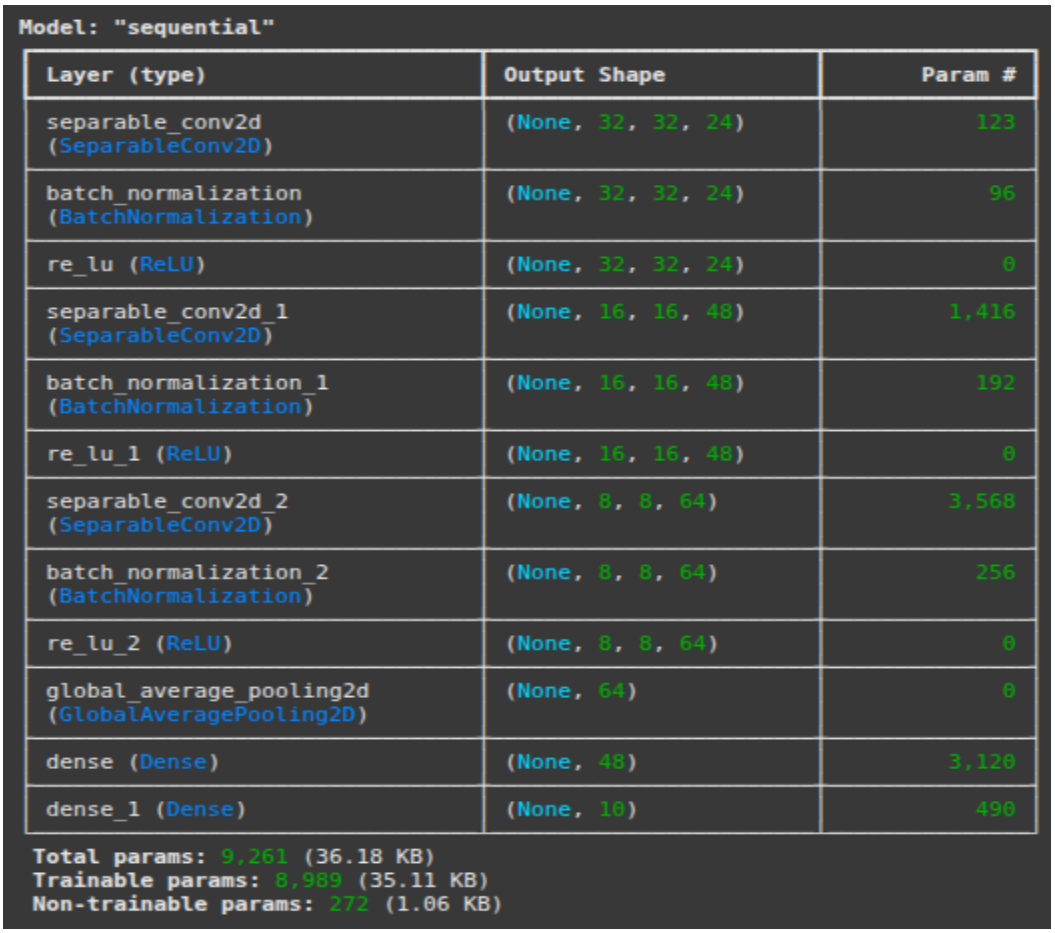
We applied dynamic post-training quantization (PTQ) on a fine-tuned MobileNetV3 model trained on the CIFAR-10 dataset. Dynamic quantization applied using PyTorch's torch.quantization.quantize_dynamic function. This targeted the nn.Linear and nn.Conv2d layers, which converts the model's weights from FP32 to INT8, while activations were quantized dynamically during inference. Model size reduced approximately 1.4 times compared to the original, with a slight improvement in inference speed, because the original model inference time was also very low, and the accuracy drop was around 5% that dynamic PTQ was effective for this classification task. However, some limitations were also observed. Dynamic quantization is most effective for linear layers, meaning convolutional layers might not benefit as significantly. Additionally, there was potential for class-specific performance degradation, particularly for classes that depend on subtle, fine-grained features.

5.1.3. Custom CNN model

First, we will start with a brief overview of the dataset. CIFAR-10 is a common dataset for image classification. There are 10 categories like airplane, frog, ship, car, cat, dog, and so on with approximately 60,000 images each with a pixel size of 32 * 32. These images are divided in such a way that 50,000 is for training purposes and 10,000 is for testing purposes. Because it has a small size of images, this dataset is great for using on edge devices.

To avoid overfitting and the model needs to be more general, we use data augmentation with random rotation, flipping, and shifting during the training process. Finally, all images are scaled between 0 and 1. Also by using one hot format the labels were changed.

For the code, we chose TensorFlow and Keras instead of PyTorch mainly because TensorFlow has more support when it comes to deploying models into edge devices like Arduino using TensorFlow Lite.



Model: "sequential"

Layer (type)	Output Shape	Param #
separable_conv2d (SeparableConv2D)	(None, 32, 32, 24)	123
batch_normalization (BatchNormalization)	(None, 32, 32, 24)	96
re_lu (ReLU)	(None, 32, 32, 24)	0
separable_conv2d_1 (SeparableConv2D)	(None, 16, 16, 48)	1,416
batch_normalization_1 (BatchNormalization)	(None, 16, 16, 48)	192
re_lu_1 (ReLU)	(None, 16, 16, 48)	0
separable_conv2d_2 (SeparableConv2D)	(None, 8, 8, 64)	3,568
batch_normalization_2 (BatchNormalization)	(None, 8, 8, 64)	256
re_lu_2 (ReLU)	(None, 8, 8, 64)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 64)	0
dense (Dense)	(None, 48)	3,120
dense_1 (Dense)	(None, 10)	490

Total params: 9,261 (36.18 KB)
Trainable params: 8,989 (35.11 KB)
Non-trainable params: 272 (1.06 KB)

Figure 5: Custom CNN Architecture

Above is the model architecture we used and below is a breakdown of each layer in the model and why it is included.

- SeparableConv2D Layers

It is a lightweight version of regular Conv2D. It splits the convolution into two parts, depthwise and pointwise, resulting in reducing the number of parameters. We used it because it saves memory and computation, making the model smaller and faster. There are three SeparableConv2D layers in the model, the first layer uses 24 filters with a 3*3 kernel, the second

one uses 48 filters with a stride of 2 for image down sampling and the third uses 64 filters to reduce the spatial size.

- Batch Normalization

The purpose of this layer is to normalize the outputs from convolutional layers. It helps our model trains faster and improves performance toward introducing weight initialization. Each SeparableConv2D layer is followed by a BatchNormalization layer.

- ReLU Activation

It adds non-linearity, which allows the model to learn complex and in-depth patterns. It is simple, fast and effective with activating only positive values, which speeds up the training process. Again, each convolutional block ends with a ReLU layer.

- GlobalAveragePooling2D

It was used instead of flattening the layer. It works by taking the average of each feature map and reducing the data to a one-dimensional vector. It's more efficient than using a flattening layer.

- Dense Layers

Adds learning capacity to the model and helps with feature extraction resulting in outputs the probability distribution over the 10 classes.

This architecture works well because it’s small under 10,000 parameters, which is excellent for Arduino deployment resulting in a balance, yet still efficiency and accuracy with each layer has been chosen carefully to reduce the size of the model.

We used the Adam optimizer and learning rate around 0.001 because our dataset has 10 different categories to predict. Also, we used categorical cross-entropy as the loss function. Then included callbacks like EarlyStopping to stop the training process if model accuracy stops improving. Also, ReduceLROnPlateau to reduce the learning rate when it's needed. Finally, the model was trained with 30 epochs approximately.

Our final stage is to put the model on the edge device like Arduino, so we saved our model in the .h5 format so we can use it later. Now we are ready to convert our model using TensorFlow Lite. We are also going to focus on quantizing the model by reducing its size by using 8-bit integers. These are the steps which are important if we want to run our model on edge devices.

At last, in this project we have built a custom CNN that can run on edge devices like Arduino that works well on the CIFAR-10 dataset for classifying multi classes. The model is fast, accurate, and reliable. By using basic techniques like data augmentation and lightweight layers we were able to make a model to balance the size and performance. Finally, it is ready to be used in real-time applications.

5.2. Methodology updated

6. Results

6.1. Development anticipated

6.1.1. Transfer Learning model

Because of the constraints of Arduino Kit, we also have to choose the small version of MobileNetV3 instead of the large version. With that being said, the transfer learning model still

got a very respectable score. In terms of classification score, macro accuracy, precision, recall are all 0.81.

Table 1: Transfer Learning Result

Metrics	accuracy	precision	recall	f1
Score	0.8102189781021898	0.8121759435717278	0.8102095904095904	0.8106668804577737

As for how lightweight the model is, we will indirectly measure it through train parameter count and FLOPs. As we are optimized to be as small as possible, we have directly replaced the last fully connected layer with the appropriate number of classes and froze everything but that layer. So, we only trained 1024 and 1 feedforward times 10 classes. That resulted in FLOPs number is around 63 million operations.

Table 2: Transfer Learning "Light" Score

Metrics	trained_param_count	flops
Score	**10250**	**62874080**

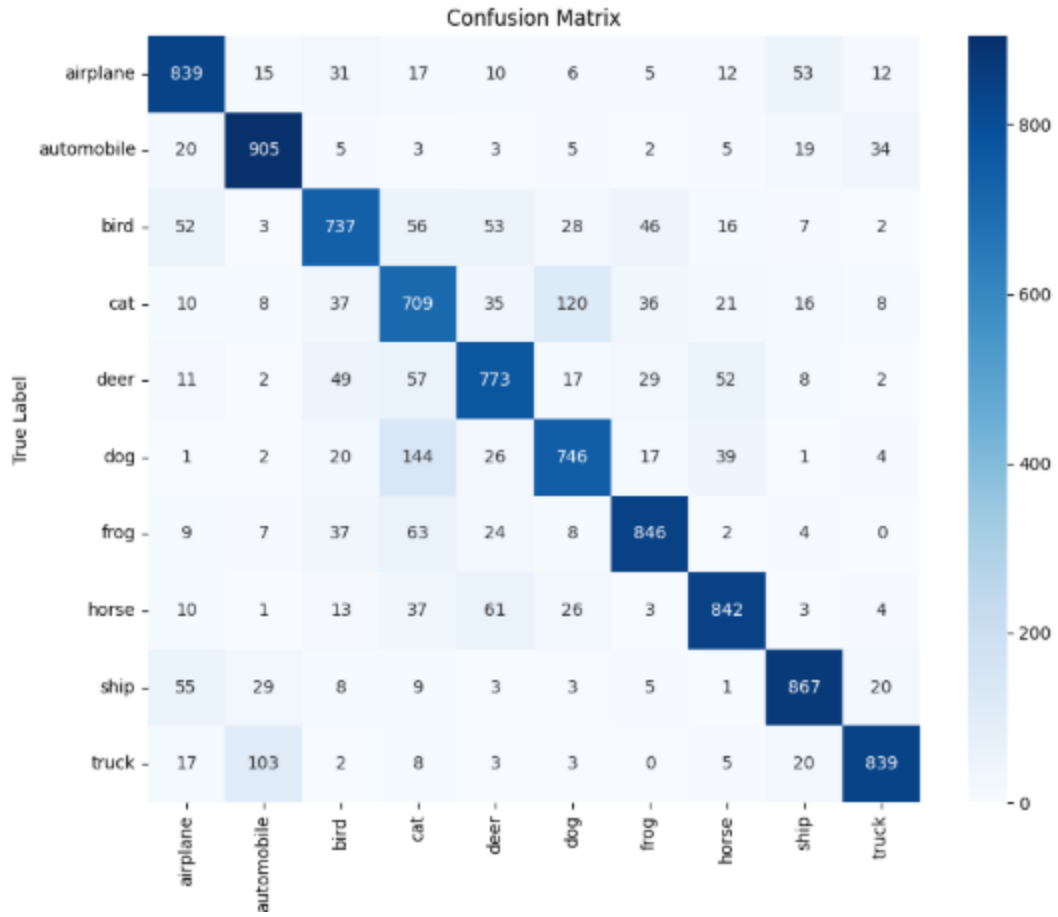


Figure 6: Transfer Learning Confusion Matrix

The biggest struggle for the model seems to be in the animal kingdom, more specifically between cats and dogs. Another big mistake is wrongly classifying trucks as automobiles. However, as the terminology of a truck could be viewed as an automobile, this type of mistake is not too grave in our opinion. Overall, the model performance is acceptable in terms of classifying and light enough that it could be loaded in some edge device.

To test the performance of the model more fairly, I have prepared a test sample of 20 images, 2 for each class, downloaded from Wikipedia. The image has horrible lighting after going through the input treatment, especially when they are dominantly blue or black in color. It confirms the

results gathered from the test set of the cifar-10 dataset. The model is struggling to determine which animal is in the picture. There is one picture of an aircraft carrier, which is classified as an airplane, which could be because of the aircraft parked on top of it

Figure 7: Transfer Learning on Test Dataset

6.1.2. Quantization

In order to make the model even smaller, we will apply quantization to the model. According to Introduction to Quantization Cooked in 🤗 With 💗 👨‍🍳, in short, "Quantization is set of techniques to reduce the precision, make the model smaller and training faster in deep learning models" (Introduction to Quantization Cooked in 🤗 With 💗 👨‍🍳, 2023). One of the easiest quantization techniques is scaling from a floating point 32 bit to an 8-bit integer. While the

representation of floating points in the model is more precise, it also takes up more space. By reducing the space each number in the model takes up, from 32 bits to 8 bits, we essentially sacrifice accuracy of the model to make it smaller. (Introduction to Quantization Cooked in 🤗 With 💗 👰, 2023, Figure 1)

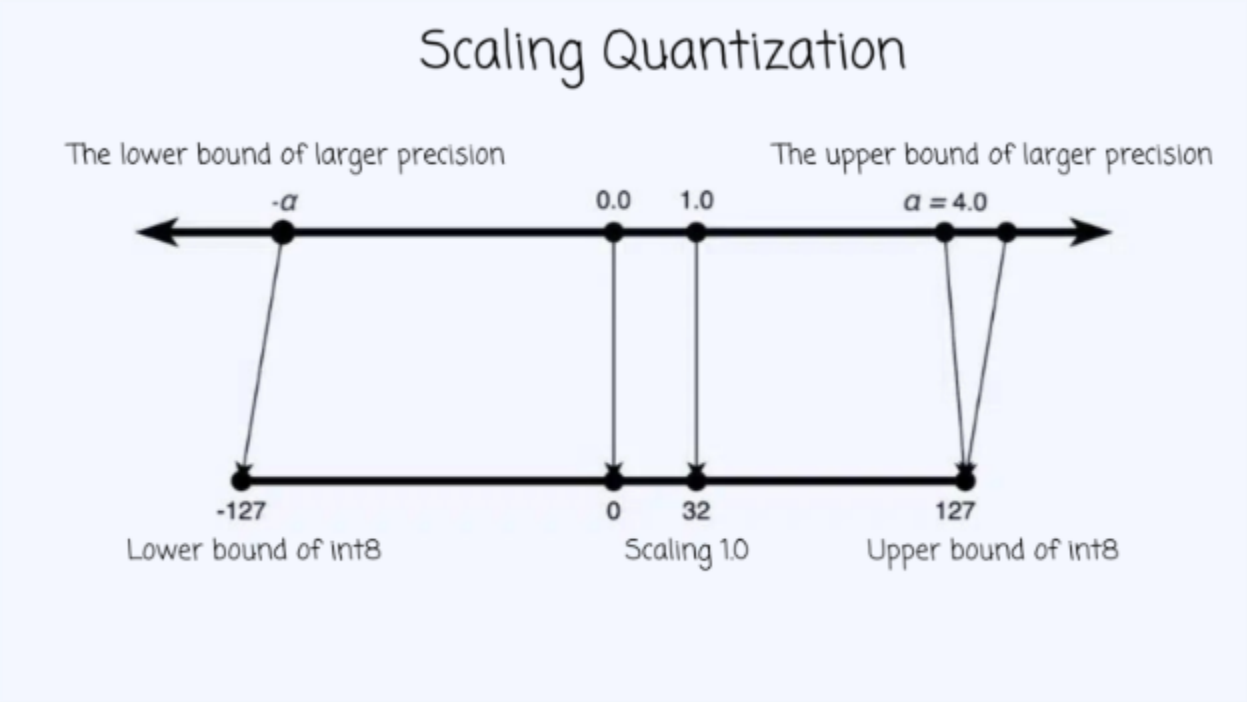


Figure 8: Scalling Quantization

This method is not without limitations. The biggest limitation is that by scaling down from to another range, any change in distribution of the before range will mess up the performance of the model after quantization a lot. For example, if the lighting of the input is changing, it will affect the predicted value of the model. In other words, models are now more susceptible to changing environments.

As for our actual model, accuracy has dropped to 0.76 (from 0.81). It is still acceptable for us at the moment, especially because F1-score is also around the same range. The most important

thing to note is that the model slims down one third of the weight, from 6200 kilobytes to 4400 kilobytes.

Table 3: After Quantization Result

Metrics	accuracy	precision	recall	f1	Model size	Model size (Quantized)
Score	**0.7635**	0.7715	0.7635	0.7617	6234.402 KB	4435.79 KB

As for the confusion matrix, they are essentially the same as before, but with more errors. It still has problems with the animal kingdom, however the problem of mislabeling airplane as ships seems to be worse than before.

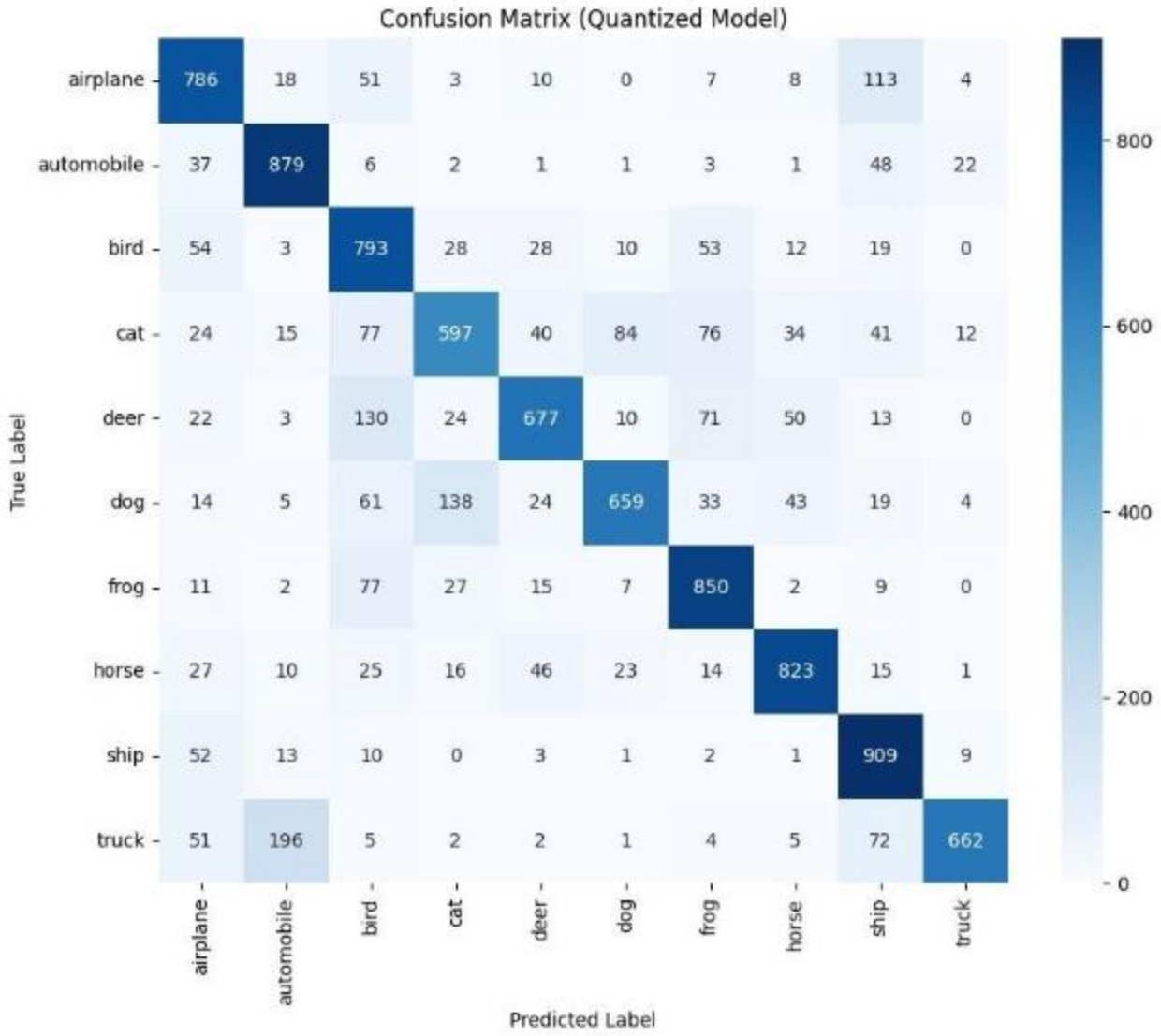


Figure 9: After Quatization Confusion Matrix

The results from our custom test sample confirm the theory that the quantized model is more susceptible to bad environments. In this experiment, all images had bad lighting, and only 4 images were labeled correctly. Interestingly, it now classifies all 2 images of dogs in the sample. In the pre-quantized model, all 2 images of a dog were classified as birds. One the flipped side, no inanimate object is labeled wrong. A lot of them are classified as animals. These results told us that we would need more preprocessing of the images before we feed them to the model.

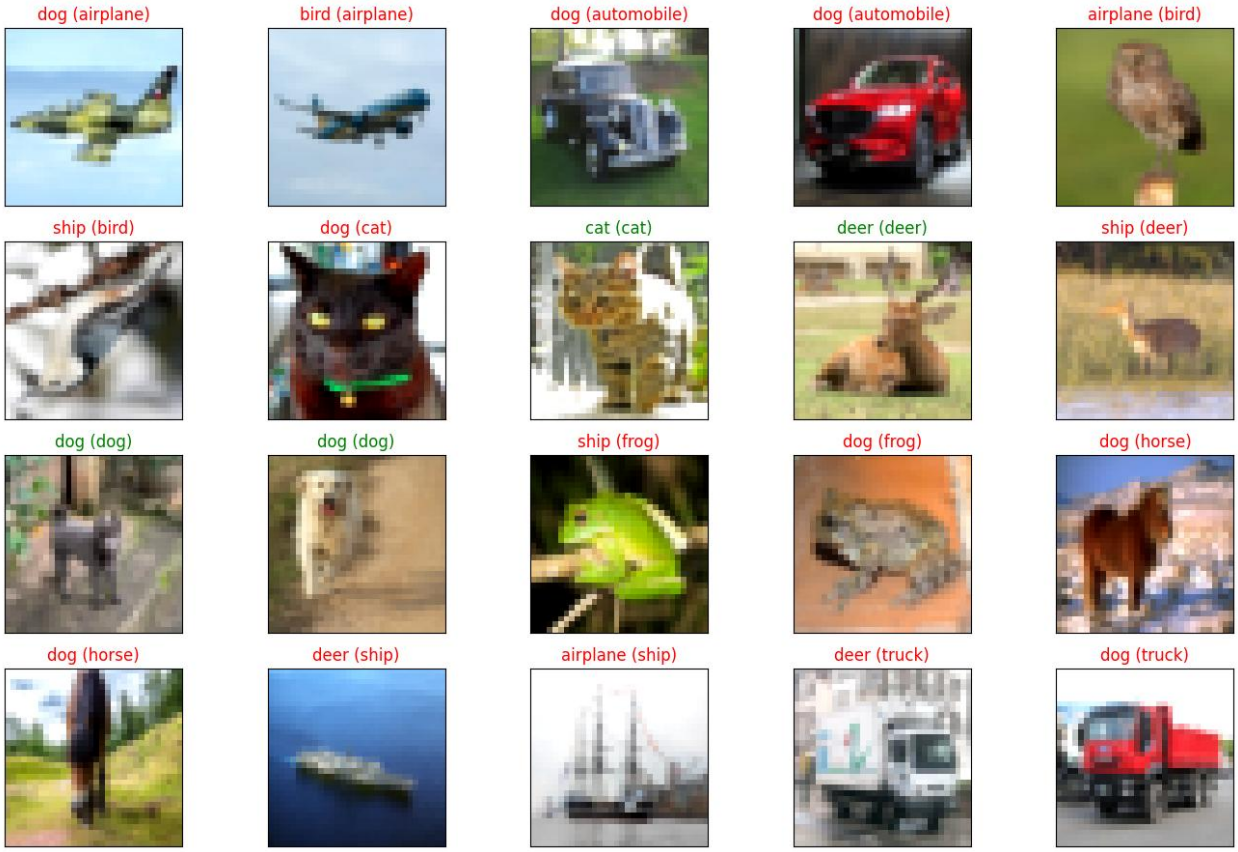


Figure 10: After Quantization on Test Dataset

6.1.3. The effect of lighting and contrast in our results

During our inference test, we found out a slight change in the condition can lead to a big swing in the result. Take the pre-quantization transfer learning model for example, if we increase lighting and contrast by 10 percent, the model accuracy drops to around 66 percent instead of 80 percent as with optimal lighting.

```
[ ]  # Define the lighting layer
     lighting_layer = LightingLayer(brightness_factor=1.1, contrast_factor=1.1)
```

Figure 11: Lighting and Contrast Layer Setting

Table 4: Lighting Effect on Model

Metrics	accuracy	precision	recall	f1
Score	0.66	0.71	0.66	0.66

The effect can also be observed in our test dataset. While the vehicle saw little change in predictions, the 2 pictures of dog class are completely different classes. The first one changed label from bird to cat, while the second one changed from bird to a ship. On the picture of the front of the horse mane, it is now correctly predicting the horse class instead of labeling it as deer.

Figure 12: Lighting Effect on Test Dataset

The effect of lighting is more serve because we currently choosing to select the highest label confidence as the final result, in real application, it is better to keep the output of the model as the probability of each class presented in the picture, we can create a more advanced function

to judge each class, if we are not seeing huge differences between a few classes, it might be better to return a list of classes instead of a winner-take-all function.

6.1.4. Our Custom CNN

After training, the model achieved 65% accuracy on the test data. Then, we used a classification report and confusion matrix to evaluate how good our model is performing for each class.

Based on the below confusion matrix, our model worked well for the classes like frog, horse, ship and truck. Also, there are some wrong predictions across the classes like cat, dog where the model fails to learn shapes and textures and airplane, ship resulting in some misclassifications as they both share similar backgrounds such as sky and ocean. That's acceptable because we have reduced the number of parameters to a huge level because our model needs to run on edge devices which results in our model does not learning in-depth features. Such as failing to predict some classes. But it does classify most trucks and automobiles clearly despite both belonging to the vehicle class. Overall, the model performed well even with a small image size and a simple layer structure.

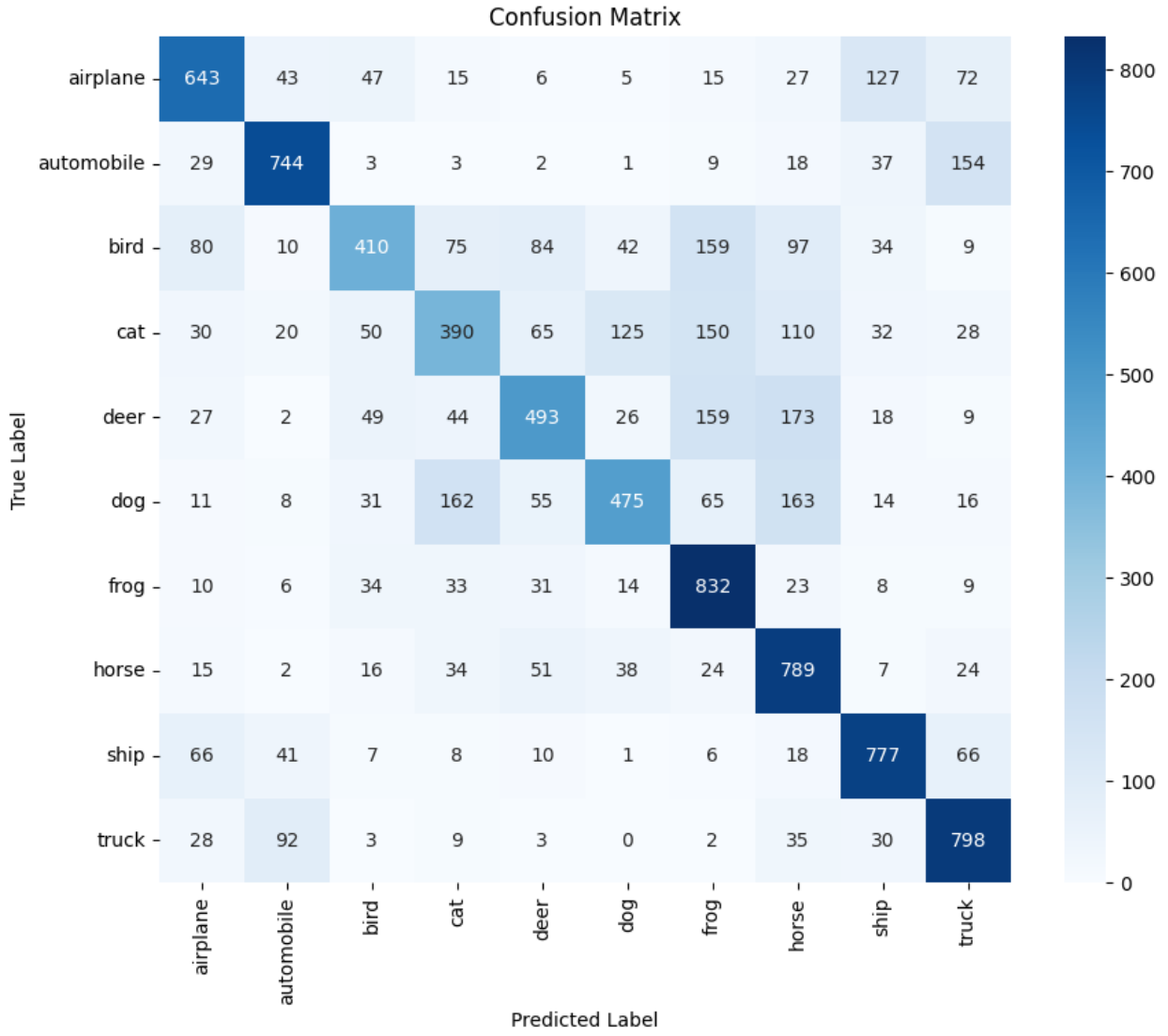


Figure 13: Custom CNN Confusion Matrix

We also test the model in a simulated inference phase, where we test our trained model with real world images to see how good they are performing. I have attached the output images below where the model predicted some real-world images.

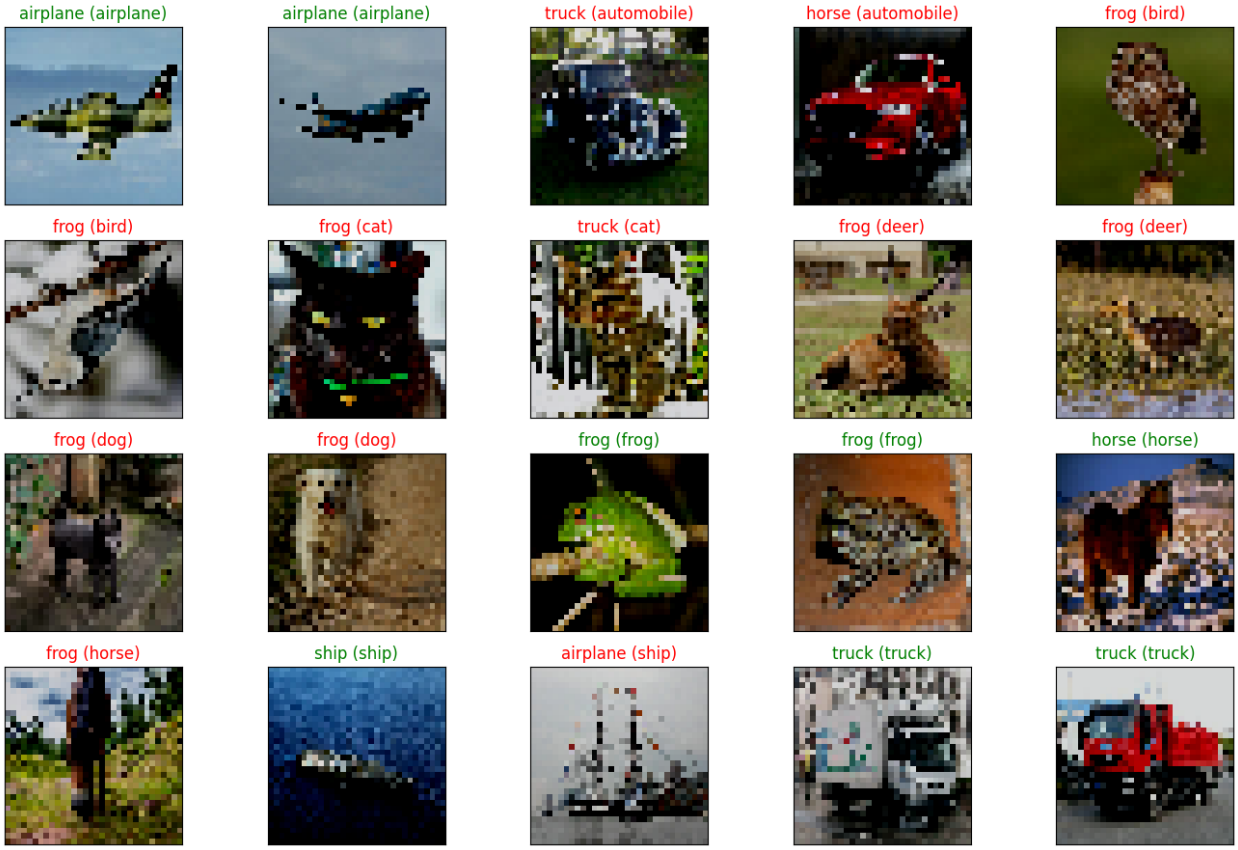


Figure 14: Custom CNN on Test Dataset

Based on the results of the outside dataset from wiki, we can see that in the animal world, our model seems to default to frogs. In the transportation objects, the model seems more confident with the side profile than a slightly slanted picture, suggesting that some level of generalization is still needed. Overall, we still think that it is an acceptable result given the constraint that we have.

Overall, the original custom CNN performed well on classes like frog, truck and horse with achieving a solid accuracy of 65% with its lightweight architecture. But the model was struggling with some classes like airplane and ship as they both share their blue backgrounds such as ocean and sky and another classes like cat and dog due to their common visual features. The overall accuracy was around 65 as the architecture was designed to be small and efficient.

Then there's the transfer learning model overall with good accuracy and predicting classes as well. Despite being optimized for low-end devices, it showed more improvements. The main reason this model benefits is from the pre-trained features, which results in strong generalization. However, it does misclassify tracks more often than other classes, indicating some trade-offs when training.

To conclude, the transfer learning model offered better overall performance and accuracy, especially in similar looking classes, however in an extreme computation limited device, like Arduino, it might be too large to deploy. As a consequence, the transfer learning model is more suited for deploying on a slightly more powerful microprocessor, such as Raspberry Pi, while our custom CNN offers acceptable result for less powerful microprocessors.

6.2. Development updated

7. Conclusions

7.1. Summary of Work

This thesis presents the planning of a custom lightweight CNN architecture amended for edge devices, specifically the Arduino TinyML platform. The project searched for the flaws of existing models like MobileNetV3, ShuffleNet and addressed them by developing a personalized

architecture that balances validity, speed, and resource efficiency. Techniques such as Quantization Aware Training (QAT) and Post Training Quantization (PTQ) were applied to further reduce model size and improve run time without major loss in precision.

7.2. Key Contributions

- Development of a unique lightweight CNN architecture adapted for small appliances
- Global appraising against popular lightweight models.
- Successful lineup and up to date inference on an Arduino ML kit.
- Quick analysis of functioning balance between model complexity and edge usability

7.3. Future Work

Expansion to support additional tasks such as object detection or multi-class classification.

Maintaining progressive optimization methods like pruning and co-training

Testing the model on varied hardware platforms to promote growth.

Integrate transfer learning to adapt the model for new datasets with refinement

7.4. Final Remarks

The outcome of this project explains architectural progression; deep learning models can be efficiently completed on limited edge devices. This covers the way for more practical, intelligent applications in the expanding sector of IoT and embedded devices.

Bibliography/References

Zhao, X., Wang, L., Zhang, Y., Han, X., Deveci, M., & Parmar, M. (2024). A review of convolutional neural networks in computer vision. *Artificial Intelligence Review*, *57*(4). https://doi.org/10.1007/s10462-024-10721-6

Sinha, S. (2024, September 3). *State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally*. IoT Analytics. https://iot-analytics.com/number-connected-iot-devices/

Iran Sarafraz, Agahi, H., & Azar Mahmoodzadeh. (2025). Convolutional neural network (CNN) configuration using a learning automaton model for neonatal brain image segmentation. PLoS ONE, 20(1), e0315538–e0315538. https://doi.org/10.1371/journal.pone.0315538

Bashir, T., Wang, H., Tahir, M., & Zhang, Y. (2025). Wind and solar power forecasting based on hybrid CNN-ABiLSTM, CNN-transformer-MLP models. Renewable Energy, 239, 122055. https://doi.org/10.1016/j.renene.2024.122055

Al-Shareeda, M. A., Yue, L., & Manickam, S. (2024). *Review of Edge Computing for the Internet of Things (EC-IoT): Techniques, Challenges and Future Directions*. *4*(1), 1–11. https://www.researchgate.net/publication/380036552_Review_of_Edge_Computing_for_the_Internet_of_Things_EC-IoT_Techniques_Challenges_and_Future_Directions

Setyawan, N., Sun, C.-C., Hsu, M.-H., Kuo, W.-K., & Hsieh, J.-W. (2025). *MicroViT: A Vision Transformer with Low Complexity Self Attention for Edge Device*. ArXiv.org. https://arxiv.org/abs/2502.05800

Elhanashi, A., Dini, P., Saponara, S., & Zheng, Q. (2023). Integration of Deep Learning into the IoT: A Survey of Techniques and Challenges for Real-World Applications. *Electronics*, *12*(24), 4925. https://doi.org/10.3390/electronics12244925

Goyal, R., Vanschoren, J., Acht, van, & Nijssen, S. (2021). *Fixed-point Quantization of Convolutional Neural Networks for Quantized Inference on Embedded Platforms*. ArXiv.org. https://arxiv.org/abs/2102.02147

API4AI. (2025, April 27). *Edge AI Vision: Deep Learning Tips | by API4AI | Medium*. Medium. https://medium.com/@API4AI/edge-ai-vision-deep-learning-on-tiny-devices-11382f327db6

Mittal, P. (2024). A comprehensive survey of deep learning-based lightweight object detection models for edge devices. *Artificial Intelligence Review*, *57*(9). https://doi.org/10.1007/s10462-024-10877-1

Elhanashi, A., Dini, P., Saponara, S., & Zheng, Q. (2023). Integration of Deep Learning into the IoT: A Survey of Techniques and Challenges for Real-World Applications. *Electronics*, *12*(24), 4925. https://doi.org/10.3390/electronics12244925

Isong, N. (2025). *Building Efficient Lightweight CNN Models*. ArXiv.org. https://arxiv.org/abs/2501.15547

Rakhee Kallimani, Pai, K., Prasoon Raghuwanshi, Iyer, S., & Alcaraz, L. (2023). TinyML: Tools, Applications, Challenges, and Future Research Directions. *ArXiv (Cornell University)*. https://doi.org/10.1007/s11042-023-16740-9

Maaz, M., Shaker, A., Cholakkal, H., Khan, S., Zamir, S. W., Anwer, R. M., & Khan, F. S. (2022, October 22). *EdgeNeXt: Efficiently Amalgamated CNN-Transformer Architecture for Mobile Vision Applications*. ArXiv.org. https://doi.org/10.48550/arXiv.2206.10589

A new lightweight network based on MobileNetV3. (2022). *KSII Transactions on Internet and Information Systems*, *16*(1). https://doi.org/10.3837/tiis.2022.01.001

Aminu Musa, Habeebah Adamu Kakudi, Mohammed Hassan, Mohamed Hamada, Usman Umar, & Maryam Lawan Salisu. (2025). Lightweight Deep Learning Models For Edge Devices—A Survey. *International Journal of Computer Information Systems and Industrial Management Applications*, *17*, 18. https://doi.org/10.70917/ijcisim-2025-0014

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2019). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. https://arxiv.org/pdf/1801.04381

Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V., & Adam, H. (2019). *Searching for MobileNetV3*. ArXiv.org. https://arxiv.org/abs/1905.02244

Ma, N., Zhang, X., Zheng, H.-T., & Sun, J. (2018). ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design. *ArXiv:1807.11164 [Cs]*. https://arxiv.org/abs/1807.11164

Jeevan, P., & Sethi, A. (2024). *Which Backbone to Use: A Resource-efficient Domain Specific Comparison for Computer Vision*. ArXiv.org. https://arxiv.org/abs/2406.05612v1

Li, Y., Zhang, W., Liu, Y., & Jin, Y. (2022). A visualized fire detection method based on convolutional neural network beyond anchor. *Applied Intelligence*.

https://doi.org/10.1007/s10489-022-03243-7

Introduction to Quantization cooked in 🤗 with 💗 🍸. (2023). Huggingface.co.

https://huggingface.co/blog/merve/quantization

Appendices

Figure 1: ShuffleNet Architecture 19
Figure 2: MobileNetV3 Cell 21
Figure 3: System Diagram 24
Figure 4: Transfer Learning Model Size 27
Figure 5: Custom CNN Architecture 29
Figure 6: Transfer Learning Confusion Matrix 33
Figure 7: Transfer Learning on Test Dataset 34
Figure 8: Scalling Quantization 35
Figure 9: After Quatization Confusion Matrix 37
Figure 10: After Quantization on Test Dataset 38
Figure 11: Lighting and Contrast Layer Setting 38
Figure 12: Lighting Effect on Test Dataset 39
Figure 13: Custom CNN Confusion Matrix 41
Figure 14: Custom CNN on Test Dataset 42

Table 1: Transfer Learning Result 32
Table 2: Transfer Learning "Light" Score 32
Table 3: After Quantization Result 36
Table 4: Lighting Effect on Model 39